密码学杂谈

汪乐平

建议安装的东西

- WSL，Windows干啥都挺麻烦的
- Python，有些简单而要求速度的程序可以用C++写
- Crypto：包含各种密码学常用工具的Python库（pip3 install pycryptodome）
- pwntools：Python库，主要是用remote
- Sage：一个包含很多东西的数学库，可以直接用，也可以在Python里调用，具体使用见文档（sudo apt install sagemath）
- CADO-NFS：一个实现NFS（数域筛法）的库，做某些特定的事情比Sage快得多
- https://gitlab.inria.fr/cado-nfs/cado-nfs
- 需要装cmake，libgmp-dev进行编译

安全密码

- 不依赖于算法的保密性，依赖于密钥的保密性
- 计算安全：通俗点说就是在可接受的时间，空间，成本内被破解概率很小的密码
- 对称加密：加解密使用相同的密钥，一般长度至少128bit
- 非对称加密：分为公钥和私钥，加解密使用不同的密钥
- 非对称加密一般依赖于数学难题

攻击手段

- 唯密文攻击：只有密文
- 已知明文攻击：有明密文对
- 选择明文攻击：能选择一些明文生成对应的密文
- 选择密文攻击：能选择一些密文生成对应的明文（肯定不会让你直接解密题目中的密文的）

取模

- 13÷3=4……1
- 13%3=1
- -1%3=2
- (a+b)%n=((a%n)+(b%n))%n
- (a-b)%n=((a%n)-(b%n))%n
- (a*b)%n=((a%n)*(b%n))%n
- 有了乘法就可以定义幂次a^m%n，m是整数（Python的pow(a,m,n)）
- 有除法吗?

一点数论知识

- a|b：b%a=0，a是b的因子
- gcd(a,b)：最大的满足c|a且c|b的正整数c
- gcd(a,b)=gcd(a-b,b)=gcd(a%b,b)，辗转相除法（欧几里得算法）
- lcm(a,b)：最小的满足a|c且b|c的正整数c
- lcm(a,b)=ab/gcd(a,b)
- 裴蜀定理：ax+by=c有整数解(x,y)的充要条件是gcd(a,b)|c

回到取模

- 设gcd(a,n)=1
- 考虑方程ax+nk=1，由裴蜀定理得存在整数解(x,k)
- a(x-n)+n(k+a)=ax+nk=1
- 存在唯一0<=x<n，ax%n=1
- x=a^-1%n，x是a的逆元
- 可以使用pow(a,-1,n)计算逆元（扩展欧几里得算法）
- a/b%n=a*(b^-1)%n
- 于是除法也有了

数论

- 互质：gcd(a,b)=1
- phi(n)：正整数n，1<=i<=n中与n互质的i的个数
- 质数：除了1和自身外没有其他正因子的正整数，phi(p)=p-1
- 合数：不是1也不是质数的正整数
- 质因数分解：对于n>1，存在唯一的一组正整数(a1,a2,...,am)和质数p1<p2<...<pm，满足n=p1^a1*p2^a2*...*pm^am
- phi(n)=n*(p1-1)/p1*(p2-1)/p2*...*(pm-1)/pm
- 欧拉定理：若gcd(a,n)=1，则a^phi(n)%n=1
- a^x%n=a^(x%phi(n))%n
- 质数定理：n以内质数密度约为1/lnn

RSA

- 终于到RSA了
- 随机生成两个很大的质数p和q，n=pq，phi(n)=(p-1)(q-1)
- 生成gcd(e,phi(n))=1
- 计算d=e^-1%phi(n)
- 公钥为(n,e)，私钥为(n,d)
- 假设明文m为满足1<=m<n且gcd(m,n)=1的整数
- 加密：c=m^e%n
- 如何解密?

RSA的难度

- 已知n,e,c=m^e%n，求m
- 如果知道phi(n)，就可以算出私钥d，进而算出m
- 如果知道p和q，就可以算出phi(n)
- 实际上对于很大的n，质因数分解是困难的
- 对于n=p*q，算phi(n)和分解n的难度是相同的（知道phi(n)可以快速分解n）
- 求d的难度和分解n是相同的（知道e和d可以很大概率快速分解n）
- 设k=ed-1，则k=k’phi(n)
- 随机1<g<n，若gcd(g,n)不互质，则找到一个质因子，否则g^k%n=1
- 取x=k/2，计算g^x%n，若仍为1则x=x/2，直到x为奇数或g^x%n!=1为止
- 若x为奇数或g^x%n=-1，则重新随机g
- 否则gcd(g^x-1,n)为n的质因子

CRT-RSA

- 中国剩余定理：给定x%n1=a1，x%n2=a2，...，且n1,n2,...两两互质则x%(n1n2...)存在且唯一
- 不互质也能做，改成%lcm(n1,n2,...)，用扩展欧几里得合并，不一定存在，存在的话唯一
- dp=d%(p-1)，dq=d%(q-1)
- 分别求出m%p，m%q后合并
- 只有dp也能分解n

分解n

- 一般n不超过256bit才能考虑直接分解n
- factordb.com
- Sage
- YAFU
- CADO-NFS
- 提示p-1/p+1光滑（质因子都很小）的话有特定的做法（不过现在没见过了）
- 所以如果除了n和e之外啥信息没有，e不是特别小，n达到512bit，基本不用考虑找RSA的漏洞（除非能选择密文攻击）

额外的等式

- 一般都是p,q,d凑出来的等式
- 一般不会太难，根据给的等式进行分析，解出需要的值
- 试着用dp来分解n吧

共模攻击

- 同一个m，同一个n，用不同的e1和e2加密，gcd(e1,e2)=1
- 裴蜀定理，解e1x+e2y=1
- m=c1^x*c2^y%n

广播攻击

- 同一个m，e非常小（比如3），多个n（至少e个）
- 用CRT合并即可

维纳攻击

- d很小（n^(1/4)），q<p<2q
- https://github.com/pablocelayes/rsa-wiener-attack
- 关键是用e/n近似e/phi(n)，做连分数展开
- 如果p和q的差更小一些，d可以更大一些（用e/(n+1-2sqrt(n))做近似）
- Cryptanalysis of RSA with Small Prime Difference

论文题

- 把别人的论文出成题
- 一般是会给一些比较奇怪的东西，想不出来有啥性质
- 基本不用考虑自己想出来（能自己想出来那就是你发paper了）
- 主要考验找论文的能力，需要知道给的东西对应的专业名词（比如最低位LSB，最高位MSB）
- （可能还有复现论文的能力）

离散对数

- c=a^b%n
- 已知a,b求c很容易（快速幂）
- 已知b,c求a很难（n=p*q）
- 已知a,c求b也很难
- 阶：质数p，1<=a<p，最小满足a^m%p=1的正整数m
- 原根：阶为p-1的数
- 一般是a=g^x%p，其中g是p的原根，求x
- BSGS：预处理g^0,g^1,...,g^(sqrt(p)-1)，用哈希表存储，枚举k=g^(xsqrt(p))，在表中找a*k^-1
- Sage有专门的离散对数函数

DSA

- 离散对数主要还是用来做数字签名
- 签名：设q是p-1的最大的质因子，随机一个k
- r=g^k%p
- s=(H(m)+xr)*k^-1%q
- (r,s)是消息m的签名
- 验签：设w=s^-1%q
- (g^H(m)*a^r)^w%p=r时验证通过

Pohlig-Hellman

- 假设p-1是光滑的，设p-1=p1^a1*p2^a2*...
- 分别计算g'=g^((p-1)/(pi^ai))和a'=a^((p-1)/(pi^ai))
- 若a'=g'^x'%p，则x'=x%(pi^ai)
- 把x'当成pi进制数，分别计算每一位（计算g'^(pi^k)和a'^(pi^k)的离散对数）
- 最后用CRT合并
- Sage内部实现

CADO-NFS

- 如果p不是很大（256bit）的话可以用CADO-NFS做离散对数
- 需要用p-1最大的质因子做预处理，第一次会比较慢，后面可以参照CADO-NFS的提示复用
- 会自己生成一个base算离散对数，需要算两次然后用换底公式

套路

- 离散对数除非有性质，否则是算不出来的
- 一般都是根据已经有的东西伪造签名
- 离散对数的数字签名一般都是用的消息哈希值做运算，如果直接把消息变成数字做运算肯定有问题，可以尝试控制消息%(p-1)的值

群

- 来点抽象的
- 定义集合G和运算*
- 封闭性：任意a,b∈G，有a*b ∈G
- 结合律：任意a,b,c∈G，有(a*b)*c=a*(b*c)
- 单位元：存在e∈G，满足任意a∈G，有a*e=e*a=a
- 逆元：任意a∈G，存在a’∈G，a*a’=a’*a=e
- 如果都满足，则称(G,*)是一个群
- 不一定有交换律
- 幂：a^0=e，a^m=a^(m-1)*a
- 于是可以扩展阶，离散对数的范围了
- 有限群：G是有限集，|G|称为群的阶
- 有限群元素的阶一定是群的阶的因子

椭圆曲线

- y^2=x^3+ax+b
- 对曲线上的点做加法，P(x1,y1)，Q(x2,y2)
- x3=m^2-x1-x2,y3=y1+m(x3-x1)
- P!=Q：m=(y1-y2)/(x1-x2)
- P=Q：m=(3x1^2+a)/2y1
- 无限群
- y^2%p=(x^3+ax+b)%p
- 有限群
- 椭圆曲线离散对数：P=kG，求k
- 应用中安全性高

常见套路

- 跟取模的离散对数差不多，
- 不过如果曲线选的不好有别的花样
- 比如群的阶正好为p的话可以尝试用smart’s attack映射到加法群
- https://github.com/jvdsn/crypto-attacks/blob/master/attacks/ecc/smart_attack.py

格

- 设v1,v2,...,vm为线性无关的n维向量
- a1,a2,...,am为整数
- v1,v2,...,vm生成的格：所有(a1v1+a2v2+...+amvm)组成的集合
- SVP：格中模长最小的向量
- CVP：格中到某特定点距离最小的向量
- 常用的两个问题
- SVP用LLL解决（Sage自带）
- CVP用Babai解决
- 很遗憾这俩我没法解释为啥是对的，真不熟
- 反正规模不大的话（一两百？）可以试试
- 基本都是论文题

CopperSmith

- 求方程f(x1,x2,...)%n=0的一个较小的根
- f(x1,x2,...)^k%(n^k)=0
- 向量每一维表示一个单项式
- f(x1,x2,...)^i0*n^(k-i0)*x1^i1*x2^i2*...作为向量
- 凑一个系数尽量小的g(x1,x2,...)%(n^k)=0

Lemma 1 (Howgrave-Graham). *Let $h(x_1, \ldots, x_n) \in \mathbb{Z}[x_1, \ldots, x_n]$ be an integer polynomial that consists of at most ω monomials. Suppose that*

(1) $h(x_1^{(0)}, \ldots, x_n^{(0)}) \equiv 0 \bmod N$ for some $|x_1^{(0)}| < X_1, \ldots, |x_n^{(0)}| < X_n$, and
(2) $||h(x_1X_1, \ldots, x_nX_n)|| < \frac{N}{\sqrt{\omega}}$.

Then $h(x_1^{(0)}, \ldots, x_n^{(0)}) = 0$ holds over the integers.

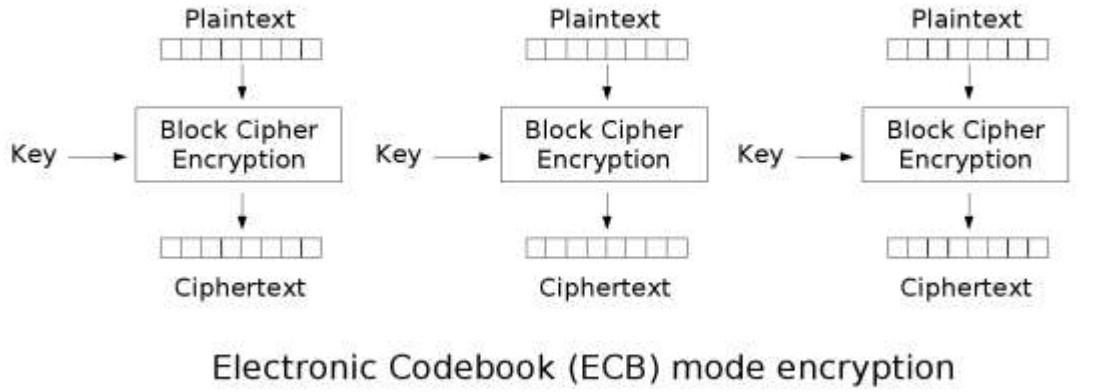
- 通俗的说就是如果g(x1,x2,...)%(n^k)=0，且x和g足够小的话，则g(x1,x2,...)=0
- A Strategy for Finding Roots of Multivariate Polynomials with New Applications in Attacking RSA Variants

分组密码

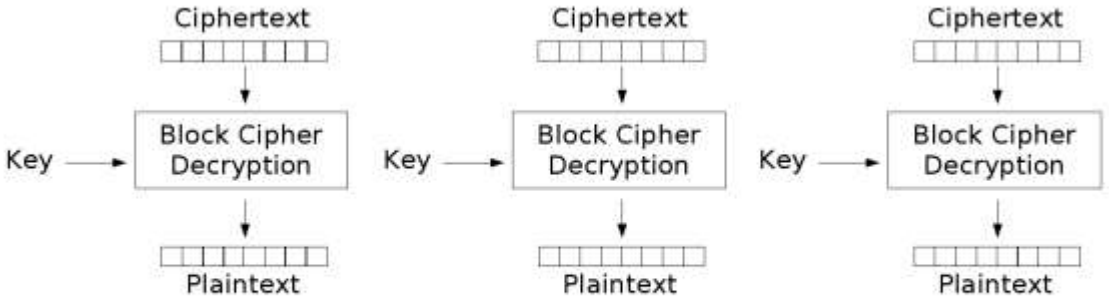
- 最常用的是AES，有时候会用SM4
- 基本都是直接调库
- 知道明密文对也弄不出密钥
- 一般都是让题目的服务端帮我们做特定消息的加密/解密
- 难点在于怎么套路服务端
- 直接攻击的几乎没有（除非是自己实现一个弱化版的）

ECB模式

- 每个块独立加解密
- 比较少见
- 各个块互相不影响，直接修改以获取加解密结果



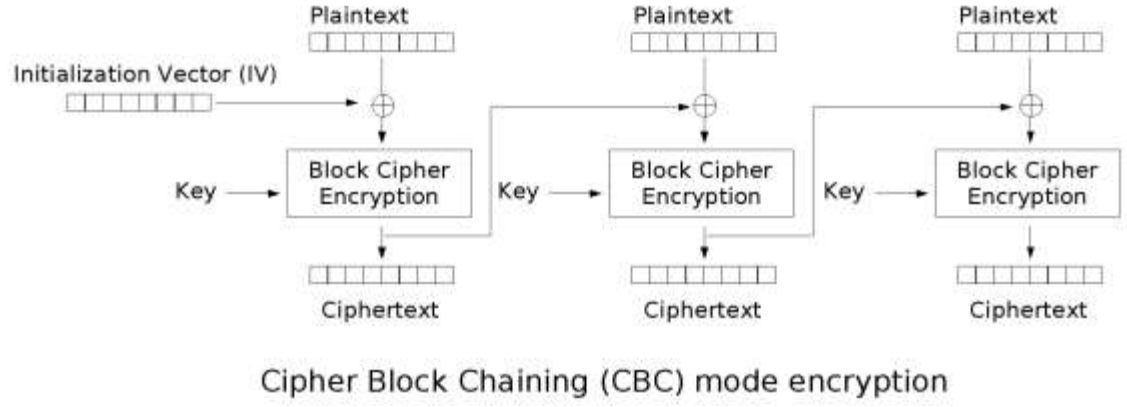
Electronic Codebook (ECB) mode encryption



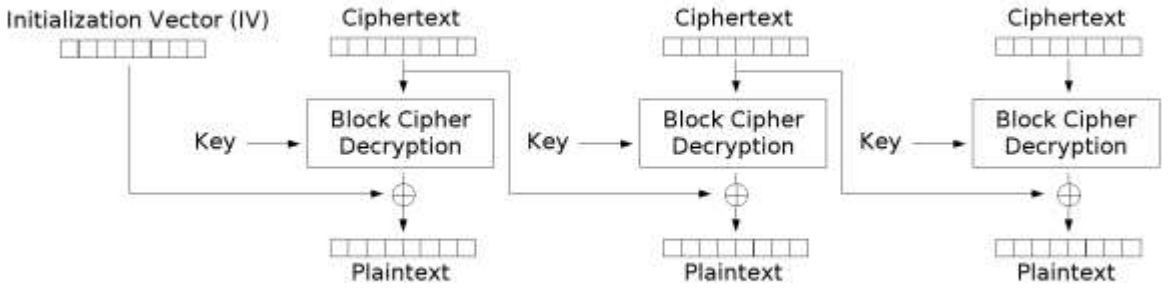
Electronic Codebook (ECB) mode decryption

CBC模式

- 每个块加密前会先异或上一个块的密文，第一个块使用额外的iv
- 可以修改iv控制第一个块的解密结果
- 或者修改两个块，控制后一个块的解密结果



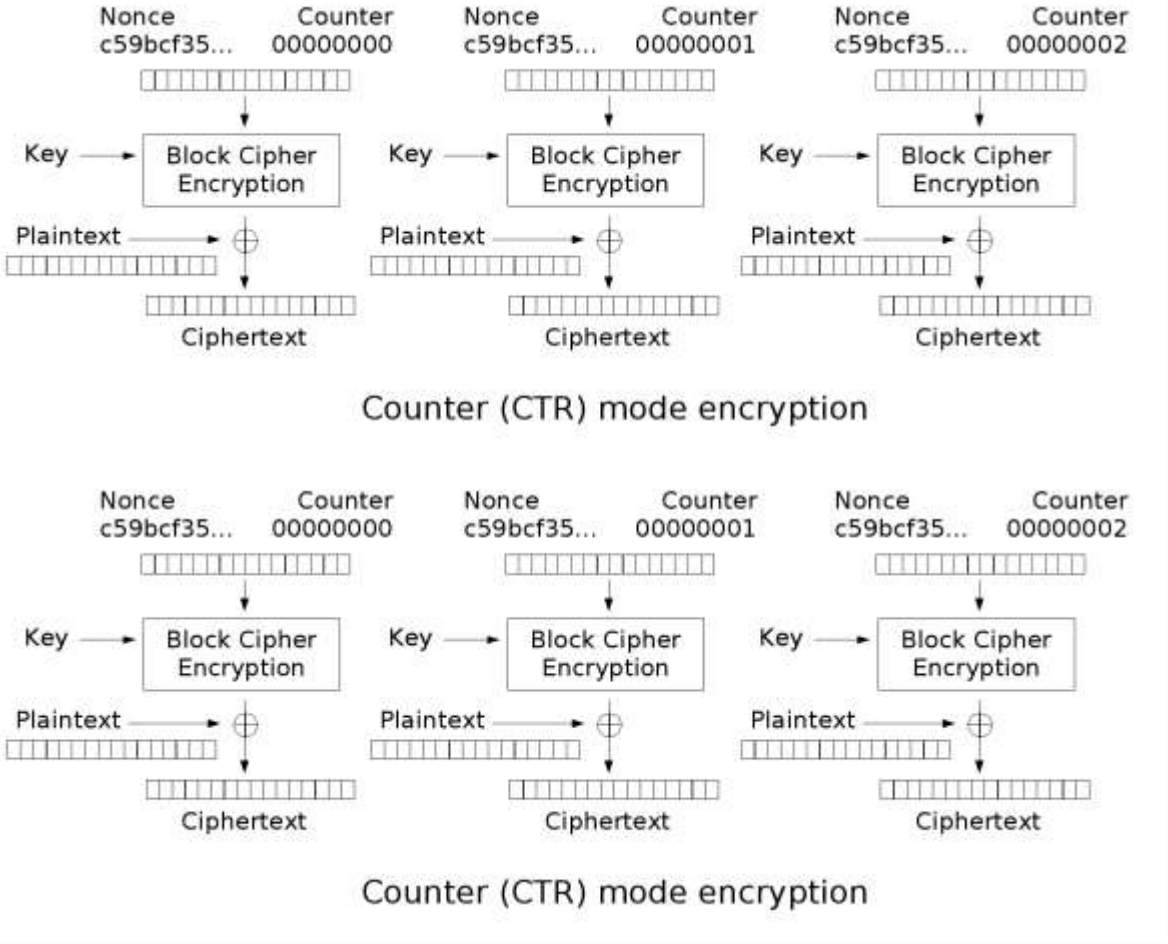
Cipher Block Chaining (CBC) mode encryption



Cipher Block Chaining (CBC) mode decryption

CTR模式

- 给定iv，将iv<<64|0，iv<<64|1，...当成128位的块，分别加密，与明文中的每个块做异或
- 可以独立控制每个块的解密结果



哈希

- 对于任意长度的消息m，生成固定长度的哈希值H(m)
- 单向函数
- 一般用于签名
- 常用的是SHA-256

不能做的东西

- 对于正常的哈希（如SHA-256，SM3）
- 生成满足特定条件的哈希值
- 生成一个跟特定m哈希值相同的m’（除了m本身）
- 生成两个哈希值相同的m，m’（除非m=m’）
- 所以关键是怎么构造出相同的m

MD5

- 但MD5是可以玩的
- 不过仅限于最后一个
- https://www.win.tue.nl/hashclash/fastcoll_v1.0.0.5.exe.zip
- 对于相同的前缀，生成两个不同的消息，MD5值相同（非常快）
- https://github.com/cr-marcstevens/hashclash
- 对于几乎相同的前缀（只在1个特定位置不同），生成两个不同的消息，MD5值相同（比较快）
- 对于几乎相同的前缀（在几个特定位置不同），生成两个不同的消息，MD5值相同（比较慢）
- 对于不同的前缀，生成两个不同的消息，MD5值相同（非常慢）
- 第四种除非只跑一次，否则时间无法接受
- 设m1,m2是长度相同且为64字节整数倍的消息，MD5(m1)=MD5(m2)，则对任意消息m，MD5(m1|m)=MD5(m2|m)
- 所以可以反复做第二种来达到特定目的

总结

- 关键要知道什么是不能做的
- 然后从剩下的里面找漏洞
- 给的东西基本不会是无用的，先找有啥性质可以用
- 遇到想不出来有啥性质的东西大概率是论文题
- 大多数Crypto题本质是数学题，需要打好数学基础（线性代数，初等数论）

THE END